

DOUBLY-CONSTRAINED NORMALIZATION IN SPIKING NETWORKS: DIRECTION, FREQUENCY AND PLACEMENT

Boyo Chen¹ Naoto Yokoya^{1,2}

¹The University of Tokyo, Tokyo, Japan

²RIKEN Center for Advanced Intelligence Project, Tokyo, Japan

ABSTRACT

Local learning rules such as spike-timing-dependent plasticity are unstable, with some neurons coming to dominate while others stop contributing. Weight normalization is the standard fix, but three of its properties are rarely separated: which side of the synapse it constrains, how often it acts, and which layer it acts on. We propose doubly-constrained normalization, which alternates a per-receiver and a per-sender constraint on consecutive updates. To separate the three properties we fix a readout-free temporal-margin R-STDP framework, vary only the normalization, and give every compared schedule one normalization operation per update. On MNIST and native-event N-MNIST the two sides are not interchangeable: postsynaptic normalization is required for the network to work, while presynaptic normalization on its own is closer to no constraint than to it. Alternating the two at the supervised decision layer exceeds postsynaptic normalization on both datasets. Diagnostics explain the asymmetry: presynaptic normalization gives every sender an equal outgoing budget, including senders that never fire, which widens the weight spectrum without improving the decision. Alternation instead keeps neurons active and concentrates the decision on a small set of senders that fire earlier for the correct class. Code and data: github.com/BoyoChen/doubly-constrained.

Index Terms— Spiking neural networks, Hebbian learning, synaptic normalization, spike-timing-dependent plasticity

1. INTRODUCTION

Local Hebbian rules train spiking networks without the global error propagation that surrogate-gradient methods use [1, 2]. Their positive feedback makes synapses compete, and the winners saturate against whatever bound the rule imposes [3]. Whether the weight distribution settles depends on how the update scales with synaptic strength [4]. Weight normalization is the standard fix. Most schemes constrain one side of the synapse. They usually bound the total input of a receiving neuron [5] and leave the weights across sending neurons free.

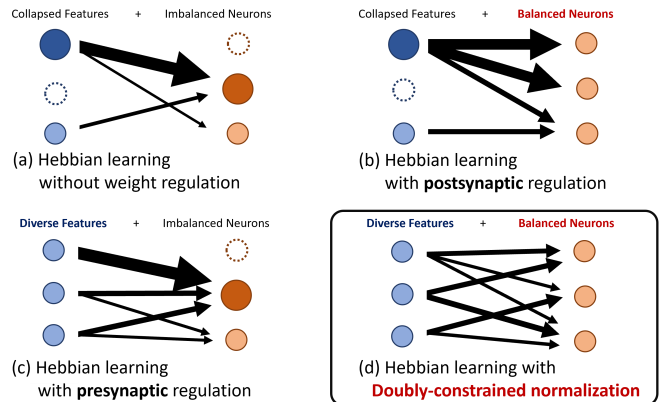


Fig. 1. The two directions a normalization can constrain. (a) Without a constraint, a few senders dominate and some receivers go unused. (b) Postsynaptic normalization bounds what each receiver holds. Receivers become balanced, but one sender still dominates. (c) Presynaptic normalization bounds what each sender distributes. Senders become balanced, but receivers stay unbalanced. (d) Doubly-constrained normalization applies both.

The two sides are not interchangeable. Postsynaptic normalization bounds what a receiver accumulates. Presynaptic normalization bounds what a sender distributes. Biological work argues that the two play complementary roles [6, 7]. Constraining both normalizes the rows and the columns of the weight matrix. Sinkhorn balancing does this globally and to convergence [8, 9]. We instead apply each constraint locally and interleave it with Hebbian updates. A schedule has three properties: which side it constrains, how often it acts, and which layer it acts on (Fig. 1).

Most work varies these three properties together. We hold the framework of [10] fixed. It is temporal-margin R-STDP. It supervises on relative spike timing and reads the prediction from output spikes. We vary only the normalization. Every compared schedule runs one normalization operation per update. Each schedule runs at the hidden layer, at the output layer, or at both.

This gives a controlled account of three things: what each

direction contributes, where the constraint must act, and how it changes the network.

2. METHOD

2.1. Substrate

We keep the framework of [10] unchanged. Neurons are single-spike integrate-and-fire. Class groups give the prediction from their output spikes, with no external readout. The hidden layer learns by local STDP. The output layer learns by temporal-margin signed LTP. Training is two-pass. The first pass computes the teaching signal. The second replays it against the pre-update state. Earliness drives threshold and amplifier homeostasis. The hidden layer freezes at a fixed epoch. The neuron and learning equations are in [10]. This paper changes only the weight constraint.

2.2. The Two Directions

Collect one layer’s weights in W , senders on rows and receivers on columns. Postsynaptic normalization fixes the total input of a receiver,

$$W_{:,j} \leftarrow W_{:,j} / \|W_{:,j}\|_1, \quad (1)$$

and presynaptic normalization fixes the total output of a sender,

$$W_{i,:} \leftarrow \rho W_{i,:} / \|W_{i,:}\|_1, \quad (2)$$

where ρ is the receiver-to-sender shape ratio. It makes the total weight comparable between the two directions. Rows and columns with zero norm stay unchanged. Applying both directions is a fixed sequence of rescalings. One pass does not satisfy both constraints. We claim no Sinkhorn convergence.

2.3. Frequency and Placement

A schedule is a pattern over consecutive learnable batch updates. We count an update per accumulated weight update, not per simulation timestep. **Q** is a step that applies (1), **P** a step that applies (2), and a dash neither. Alternating **P** and **Q** runs one operation per update, the same as either direction alone. This separates which side is constrained from how often it is constrained. Each schedule runs at the hidden layer, at the output layer, or at both. This separates the effect of a constraint from its location.

3. EXPERIMENTS

We ask two questions. Does constraining the presynaptic side help a direct spike decision? Where must the constraint act?

Protocol. MNIST uses DoG preprocessing and 40-step rank-order encoding. N-MNIST [11] uses native ON/OFF events in 20 bins. The hidden layer freezes at epoch 2.

Table 1. Normalization schedule. Test accuracy (%), $n = 10$ paired seeds. Every schedule runs at both the hidden and the output layer. Brackets give the paired 95% interval of the difference from post only, in percentage points. An interval is coloured only when it excludes zero. Table 2 then varies which layer runs the schedule.

Schedule	Pattern	MNIST	N-MNIST
Post only	Q Q Q Q	97.98	97.13
Post only, 1/2 freq	- Q - Q	97.72 [-0.36, -0.16]	97.04 [-0.20, +0.02]
Pre only	P P P P	66.12 [-34.0, -29.7]	44.73 [-54.1, -50.8]
Pre only, 1/2 freq	P - P -	63.92 [-36.4, -31.7]	45.85 [-52.3, -50.3]
No constraint	- - - -	53.10 [-45.4, -44.4]	19.16 [-79.5, -76.4]
Doubly, alternating	P Q P Q	98.14 [+0.10, +0.23]	97.44 [+0.24, +0.39]

P = pre-normalization; Q = post-normalization; - = none. The four-update pattern repeats. Each half-frequency row follows the full-frequency schedule it halves.

Table 2. Layer placement. Test accuracy (%), $n = 10$ paired seeds. The first two columns give the schedule each layer runs. Brackets give the paired 95% interval of the difference from the first row (pp).

Hidden	Output	MNIST	N-MNIST
Post	Post	97.98	97.13
Doubly	Post	97.84 [-0.22, -0.05]	96.80 [-0.44, -0.23]
Post	Doubly	98.23 [+0.18, +0.32]	97.46 [+0.25, +0.41]
Doubly	Doubly	98.14 [+0.10, +0.23]	97.44 [+0.24, +0.39]

Every condition runs 20 epochs at ten fixed seeds. We pair each difference over those seeds. Its 95% Student- t interval excludes zero unless the text calls the result a tie. Code, configurations and the analysis that regenerates Tables 1 to 4 and Figs. 2 to 4 are archived at doi.org/10.5281/zenodo.22875127.

3.1. Which Constraint, and How Often

A constraint is necessary. Removing it costs 44.88 percentage points (pp) on MNIST and 77.97 pp on N-MNIST. Direction then decides how much it helps. Pre alone is closer to no constraint than to post. It loses 31.85 and 52.41 pp. Alternating the two beats post by +0.165 and +0.312 pp. Frequency does not explain this gain. Alternating runs post at half the rate post only does, and halving post alone costs 0.261 pp on MNIST and ties on N-MNIST. Alternating is half-rate post with presynaptic steps in the empty slots. Those steps are worth +0.426 and +0.402 pp.

3.2. Where the Constraint Belongs

Doubly at the hidden layer alone does not help. It costs 0.136 pp on MNIST and 0.333 pp on N-MNIST. At the output layer alone it gains +0.250 and +0.328 pp. Applying it at both layers costs 0.085 pp against the output layer alone on MNIST and ties on N-MNIST. The constraint works only at the supervised decision layer. Plastic lifetime does not explain this (Table 3). We gave both layers the same number of plastic

Table 3. Placement against plastic lifetime. Paired difference between doubly at the output layer and doubly at the hidden layer (pp), with its 95% interval. The first row is the schedule used everywhere else, at the ten seeds of Table 2. The other two give both layers the same plastic span, at four seeds. On N-MNIST with twenty epochs each, every network falls silent by epoch 4, so there is nothing to compare.

Epochs plastic		Output-only – hidden-only	
Hidden	Output	MNIST	N-MNIST
2	20	+0.39 [+0.28, +0.49]	+0.66 [+0.62, +0.71]
20	20	+1.16 [+0.82, +1.51]	silent
2	2	+6.58 [+5.15, +8.01]	+0.01 [−1.98, +2.00]

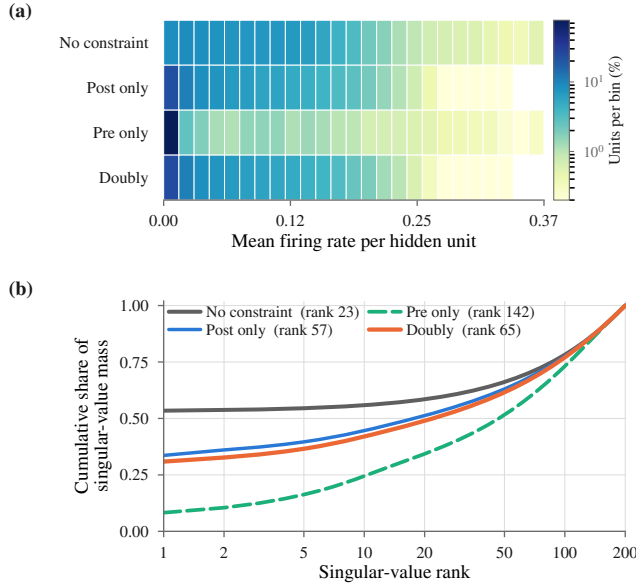


Fig. 2. What each schedule does to the network. MNIST, epoch 20, means over ten seeds, for the four every-batch schedules of Table 1, each applied at both layers. (a) Distribution of mean firing rate across the 7500 hidden units. Colour gives the percentage of units per bin on a logarithmic scale. (b) Cumulative share of singular-value mass in the output weight matrix against rank. The legend gives each schedule’s effective rank.

epochs. The output layer still wins on MNIST at both spans. On N-MNIST the control gives no answer. Leaving the hidden layer plastic silences the network, and at two epochs the two placements tie.

3.3. What the Schedules Do

These schedules run at both layers, so they also act on the hidden layer. The two unusable schedules fail in opposite ways. Without a constraint the hidden layer is the most active of the four, and the output weight has the lowest effective rank. High

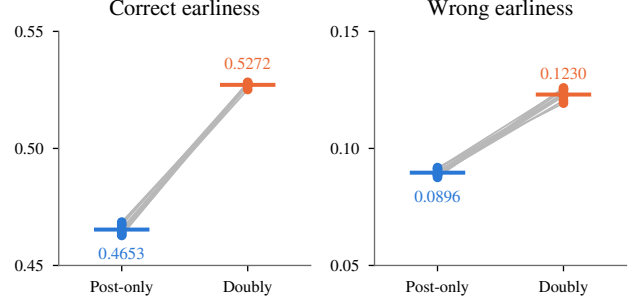


Fig. 3. Correct and wrong evidence timing on native N-MNIST. Post-only versus doubly at epoch 20. One grey line per seed, ten paired seeds. The horizontal tick is the condition mean. Both panels span 0.10, so the two slopes are comparable.

Table 4. N-MNIST decision and evidence summary. Native events, epoch 20, $n = 10$ paired seeds, test set. Earliness is normalized so that larger means earlier. Parentheses give the paired difference in pp.

Metric	Post-only	Doubly
Accuracy (%)	97.131	97.459 (+0.328)
No decision (%)	0.181	0.030 (−0.151)
Correct earliness (%)	46.534	52.716 (+6.182)
Wrong earliness (%)	8.964	12.302 (+3.338)
Evidence gap (%)	37.570	40.414 (+2.844)

activity alone is not a richer representation. Pre-only does the reverse. It gives every sender the same outgoing budget, including hidden units it has silenced. The result is the widest spectrum at the lowest activity. Spectral spread shows how a constraint distributes weight, not how well the layer performs. Doubly keeps the narrow activity band of post-only and shifts only slightly towards the spread of pre-only. It is the only one of the four that gains accuracy.

3.4. How the Decision Moves in Time

Earliness measures how strongly a class is driven. With single-spike neurons, the faster a class accumulates input, the sooner it fires. We measure it separately for the correct class and for the strongest wrong class. Correct earliness advances by +6.182 pp and the strongest wrong class by +3.338 pp. The gap therefore widens by +2.844 pp. The whole evidence distribution moves earlier. Doubly does not suppress wrong evidence. It advances correct evidence more.

3.5. Which Senders Carry the Decision

Both arms continue from one shared checkpoint. Their hidden layers stay identical throughout, so they differ only in how they use one fixed feature set. For each sample we score

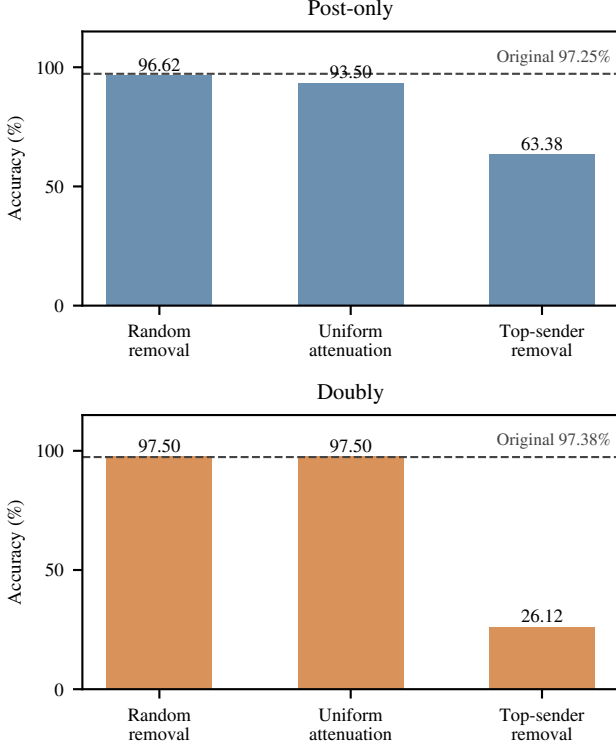


Fig. 4. Sender-removal intervention on MNIST. Accuracy after three interventions on the senders feeding the decision layer. Epoch 20, four paired seeds, 200 validation diagnostic samples per seed rather than the full test set. Dashed lines mark unperturbed accuracy. Top-sender removal deletes the highest-scoring 10% of a sample’s positive-support senders. Random removal deletes as many from the same pool. Uniform attenuation scales every sender so that the same total support is withdrawn. Random and uniform withdrawal leave doubly unchanged.

every sender by its weighted positive support at the decision time,

$$S_i^+ = \frac{1}{|g|} \sum_{j \in \mathcal{F}} \max(0, a_i(t_j) w_{ij}) e_j, \quad (3)$$

where $\mathcal{F}$ are the fired receivers of the correct class, t_j is receiver j ’s first spike, e_j its earliness, and $|g|$ the class group size.

Withdrawing support uniformly costs post-only 3.75 pp and leaves doubly unchanged. Doubly therefore tolerates a uniform change in synaptic gain, which is what device-level drift looks like. Withdrawing the same total from a sample’s top senders costs post-only 30.1 pp and doubly 71.4 pp. Neither arm produces no-decision outputs. Under doubly the loss happens because wrong classes overtake the correct one. Doubly therefore concentrates the decision on a small set of senders. That fragility is the cost of the specialization, not a

Table 5. Published N-MNIST results. Test accuracy (%). *Local learning*: every weight update is local and the decision is read from output spikes. *SVM readout*: local STDP features with a globally trained linear classifier on top. *Surrogate gradient*: the spike derivative is replaced by a smooth approximation and error is backpropagated through time. Coding, network size and training budget differ across rows. The table positions our result rather than controlling for them.

Method	Type	N-MNIST
rTSTD [12]	local learning	89.3
Doubly (ours)	local learning	97.5
Safa et al. [13]	SVM readout	99.3
SLAYER [14]	surrogate gradient	99.2

robustness result.

3.6. Comparison with Published N-MNIST Results

Two methods in Table 5 score higher. An SVM readout on STDP features reaches 99.3%, and surrogate-gradient training reaches 99.2%. The only local-learning N-MNIST result we could verify from its source is 89.3%.

MNIST goes the other way. A supervised STDP classification layer on a locally trained feature extractor reaches 98.9–99.2% [15]. Our method reaches 98.2% in Table 2. Those networks are local throughout and decide on the first spike. They report no N-MNIST result. Earlier local-learning MNIST results are lower [16, 17].

Limits. Layer position, function and update rule differ together. The placement result therefore does not isolate supervision as the cause. Table 3 removes plastic lifetime from that list on MNIST, but not on N-MNIST. We report aggregate evidence shifts and do not separate corrected errors from newly introduced ones. Each model selects its own sender set, so the two intervention arms are not identical. The removal result bounds how much the decision depends on sender identity given frozen features. It says nothing about feature quality.

4. DISCUSSION AND CONCLUSION

Normalization has two separable functions in this framework. Postsynaptic normalization keeps the network usable. Presynaptic normalization redistributes influence across senders, but on its own it is not a usable constraint. Of the schedules we tested, only alternating the two gains accuracy at a matched operation count. It gains only when the supervised decision layer applies it. It works by concentrating the decision on a small set of senders that fire earlier for the correct class. The evidence covers MNIST and native N-MNIST under one architecture and one training budget. Two questions stay open: does the rule hold in deeper networks, and does the specialization survive input perturbation?

5. REFERENCES

- [1] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke, “Surrogate gradient learning in spiking neural networks,” *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [2] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, pp. 3625, 2020.
- [3] Sen Song, Kenneth D. Miller, and L. F. Abbott, “Competitive hebbian learning through spike-timing-dependent synaptic plasticity,” *Nature Neuroscience*, vol. 3, pp. 919–926, 2000.
- [4] Mark C. W. van Rossum, Guo-qiang Bi, and Gina G. Turrigiano, “Stable hebbian learning from spike timing-dependent plasticity,” *Journal of Neuroscience*, vol. 20, no. 23, pp. 8812–8821, 2000.
- [5] Timoleon Moraitis, Dmitry Toichkin, Adrien Journé, Yansong Chua, and Qinghai Guo, “SoftHebb: Bayesian inference in unsupervised Hebbian soft winner-take-all networks,” *Neuromorphic Computing and Engineering*, vol. 2, no. 4, pp. 044017, 2022.
- [6] Rui Ponte Costa, Beatriz E. P. Mizusaki, P. Jesper Sjöström, and Mark C. W. van Rossum, “Functional consequences of pre- and postsynaptic expression of synaptic plasticity,” *Philosophical Transactions of the Royal Society B*, vol. 372, no. 1715, pp. 20160153, 2017.
- [7] Yoonsuck Choe, Risto Miikkulainen, and Lawrence K. Cormack, “Effects of presynaptic and postsynaptic resource redistribution in hebbian weight adaptation,” *Neurocomputing*, vol. 32–33, pp. 77–82, 2000.
- [8] Richard Sinkhorn and Paul Knopp, “Concerning non-negative matrices and doubly stochastic matrices,” *Pacific Journal of Mathematics*, vol. 21, no. 2, pp. 343–348, 1967.
- [9] Philip A. Knight, “The sinkhorn–knopp algorithm: Convergence and applications,” *SIAM Journal on Matrix Analysis and Applications*, vol. 30, no. 1, pp. 261–275, 2008.
- [10] Boyo Chen and Naoto Yokoya, “Temporal-Margin R-STDP for Readout-Free Spiking Neural Networks,” in *Int. Conf. Neural Information Processing (ICONIP)*, 2026, Accepted for publication.
- [11] Garrick Orchard, Ajinkya Jayawant, Gregory K. Cohen, and Nitish Thakor, “Converting static image datasets to spiking neuromorphic datasets using saccades,” *Frontiers in Neuroscience*, vol. 9, pp. 437, 2015.
- [12] Kedir Dawud Muleta and Bai-Sun Kong, “Rate-coded target-modulated STDP learning for efficient spiking neural networks,” *IEEE Access*, vol. 14, 2026.
- [13] Ali Safa, Ilja Ocket, André Bourdoux, Hichem Sahli, Francky Catthoor, and Georges Gielen, “A new look at spike-timing-dependent plasticity networks for spatio-temporal feature learning,” *arXiv:2111.00791*, 2022.
- [14] Sumit Bam Shrestha and Garrick Orchard, “SLAYER: Spike layer error reassignment in time,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [15] Gaspard Goupy, Pierre Tirilly, and Ioan Marius Bilasco, “Neuronal competition groups with supervised STDP for spike-based classification,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [16] Peter U. Diehl and Matthew Cook, “Unsupervised learning of digit recognition using spike-timing-dependent plasticity,” *Frontiers in Computational Neuroscience*, vol. 9, pp. 99, 2015.
- [17] Milad Mozafari, Mohammad Ganjtabesh, Abbas Nowzari-Dalini, Simon J. Thorpe, and Timothée Masquelier, “Bio-inspired digit recognition using reward-modulated spike-timing-dependent plasticity in deep convolutional networks,” *Pattern Recognition*, vol. 94, pp. 87–95, 2019.